

Lottery Search System

01 โจทย์ & คำตอบ

DATASET

ticket 1,000,000 ใบ ตั้งแต่ 000000 ถึง 999999 (6 หลัก) ค้นหา wildcard เช่น ****23 , 1****5 , 123***

SEARCH

REQUIREMENT ที่สำคัญที่สุด

คนสองคนค้น pattern เดียวกันพร้อมกัน ห้ามได้ ticket ใบเดียวกันเด็ดขาด

Pool ที่ match 123*** : 123456 123789 123999

User A ค้น 123*** → 123456 User B ค้น 123*** (เวลาเดียวกัน) → 123456 → 15 123789 แทน

Database

เลือก DB อะไร — PostgreSQL / Mongo / Redis / Elastic / ClickHouse แล้วอธิบายเหตุผล

คำตอบ

PostgreSQL — SELECT ... FOR UPDATE SKIP LOCKED แก้ concurrency ได้ตรงจุดโดยไม่ต้องมี distributed lock แยก, generated column + B-tree รองรับ wildcard ได้เร็ว

Algorithm

Wildcard search หาอย่างไร — Prefix Index, Trie, Bitmap, Inverted Index, B-tree, Hash (ไม่ใช่แค่ LIKE '%23' เพราะข้อมูล 1 ล้านแถว)

คำตอบ

แยกเก็บตัวเลขแต่ละหลักของ ticket เป็นคอสัมน์ของตัวเอง (d0..d5) แล้วสร้าง index ให้ทุกคอสัมน์ เวลาค้นด้วย pattern ที่มีบางหลักระบุตัวเลขจริง Postgres จะเลือกใช้ index ของหลักนั้นให้เองโดยอัตโนมัติ ทำให้หาเร็วโดยไม่ต้องสร้างโครงสร้างข้อมูลพิเศษเพิ่มเติม

concurrency

กัน race condition อย่างไร — SELECT ... FOR UPDATE SKIP LOCKED, Redis Lock, หรือ Reservation (available → reserved → sold แบบ atomic)

คำตอบ

SELECT ... FOR UPDATE SKIP LOCKED ในหนึ่ง transaction ล็อก ticket หนึ่งทีตอนค้นเจอ กันสองคนได้ใบเดียวกัน แล้วผูกกับ reservation lifecycle ด้านล่างเพื่อไม่ให้ ticket หายจาก pool ถ้าค้นแล้วไม่ซื้อ

available

 →

reserved (5 นาที)

 →

sold

หมดเวลา 5 นาที ยังไม่ซื้อ

Performance

วิเคราะห์ trade-off — search complexity, index, memory (เช่น Trie เร็วมากแต่ใช้ RAM เยอะ)

คำตอบ

ยิ่งรู้ตัวเลขในหลักไหนแน่นอนมากเท่าไร ก็ยิ่งค้นเจอเร็วขึ้น เพราะ Postgres ใช้ index ของหลักนั้นตัดตัวเลือกที่ได้ทันที ส่วน pattern ที่เป็น * ล้วน (ไม่รู้เลยสักหลัก) ต้องไล่เช็คทุกแถว แต่ก็ยังเร็วพอเพราะข้อมูลมีแค่ 1 ล้านแถว

รู้ 2 หลักขึ้นไป

 เร็วสุด

รู้ 1 หลัก

 เร็ว

ไม่รู้เลย (*****)

 ช้าสุด (น่าอึดใจ)

เลือกใช้ B-tree index เพราะเร็วพอสำหรับทุกกรณีข้างบน และกินความจำน้อย ไม่ต้องเก็บโครงสร้างข้อมูลพิเศษไว้ใน RAM ตลอดเวลา

02 Future Improvements

แนวทางขยายต่อถ้าระบบโตขึ้นหรือมี traffic จริงมาวัดผล — ยังไม่ต้องทำตอนนี้

- แยกเครื่อง Server รับเฉพาะการค้นหา — เพิ่มเครื่อง database สืบรองที่คอยก๊อปปี้ข้อมูลจากเครื่องหลักตลอดเวลา แล้วให้การค้นหาที่ยังไม่ถึงขั้นจอง ไปถามเครื่องสำรองนี้แทน ส่วนเครื่องหลักรับหน้าที่แค่ตอนต้อง "จองจริง" เท่านั้น เพราะในระบบจริงจำนวนคนค้นหาจะเยอะกว่าคนที่กดจองมาก การแยกแบบนี้ช่วยให้รองรับคนค้นหาพร้อมกันได้มากขึ้น โดยไม่ไปแย่งทรัพยากรกับเครื่องหลักที่ต้องดูแลเรื่องจองและ lock ส่วนข้อมูลที่เปลี่ยน (เช่น ticket ถูกจองไปแล้ว) จะไปกลับจากเครื่องหลักไปอัปเดตเครื่องสำรองให้อัปเดตในมิติผ่าน replication ของ PostgreSQL เอง ไม่ต้องเขียนโค้ด sync เอง แต่จะมี delay เล็กๆ ระดับมิลลิวินาที
- แบ่งข้อมูลเป็นก้อนตามเลขหลักแรก — ถ้าจำนวน ticket โตขึ้นมากๆ (เกิน 10 ล้านใบ) แบ่งข้อมูลออกเป็น 10 ก้อนตามเลขหลักแรกของ ticket ช่วยให้แต่ละก้อนมีขนาดเล็กลงและกระจายโหลดได้ pattern ที่ไม่ได้ระบุเลขหลักแรก เช่น ****23 ต้องกระจายไปถามทุกก้อนแล้วรวมผลลัพธ์เอง ซึ่งช้ากว่าการค้นหาในก้อนเดียว เหมาะทำตอนข้อมูลโตจริงจนเครื่องเดียวรับไม่ไหวเท่านั้น
- เก็บสถิติก่อนตัดสินใจเพิ่ม cache — ดูว่าคนค้นหาแบบไม่รู้เลขเลยสักหลักเกิดขึ้นบ่อยแค่ไหนจริง ถ้าบ่อยตามคาดไม่ทำอะไรเพิ่ม ถ้าบ่อยค่อยสร้างระบบ cache ช่วยตามที่พูดถึงใน section performance

03 Wildcard Pattern Matching

ตัวอย่าง pattern 123*** เทียบกับ ticket ตัวอย่างในระบบ และ WHERE clause ที่แปลงได้

1 2 3 * * *

SELECT id, number FROM tickets WHERE d0='1' AND d1='2' AND d2='3' AND NOT allocated LIMIT 1;

123456 match

123789 match

123999 match

998823 no match

997723 no match

555523 no match

444423 no match

04 Preventing Duplicate Allocation

Requirement ที่สำคัญที่สุด: User A และ User B ค้น pattern เดียวกัน (123***) พร้อมกัน ผลลัพธ์ด้านล่างคือสิ่งที่เกิดขึ้นจริงเมื่อใช้ SELECT ... FOR UPDATE SKIP LOCKED

candidates ที่ตรง pattern: 123456, 123789, 123999

USER A

123456

locks 123456 -> allocated = TRUE -> COMMIT

USER B

123789

123456 is locked -> SKIP LOCKED -> picks next available: 123789 -> allocated = TRUE -> COMMIT

05 Ticket Lifecycle

ticket ไม่ได้ถูก mark ว่า allocated ถ้าวรตอน search — มี state ดันกลางเพื่อกัน ticket หายจาก pool ถ้า user ค้นแล้วไม่ซื้อ ภาพด้านล่างคือ ticket ที่อยู่ในสถานะ "reserved" อยู่

● available → ● reserved → ● sold

reservation จริงมีเวลา 5 นาที ถ้าไม่ซื้อภายในเวลานี้ ticket จะกลับเป็น available ให้คนอื่นค้นเจอได้ใหม่โดยอัตโนมัติ

สมมติในเวลาเดียวกันนี้ User B ค้นหา pattern เดียวกัน แล้วระบบเจอ ticket ใบนี้พอดี — ผลลัพธ์ที่เกิดขึ้นจริง:

ติดล็อกอยู่กับ User A - SELECT ... FOR UPDATE SKIP LOCKED ทำให้ User B ข้ามใบนี้ไปเจอใบถัดไปแทน ไม่มีทางได้ใบเดียวกันแน่นอน